This Python implementation provides a **generic, modular framework** for solving the N-Puzzle problem using uninformed search strategies. The key design principle is **separation of concerns**: the search algorithms (DFS, BFS, UCS) are decoupled from the problem-specific logic (N-Puzzle), making the codebase reusable for other state-space search problems.

Code Structure Analysis

1. Generic Uninformed Search Function

```
def uninformed_search(start_state, goal_test_fn, successor_fn,
                      cost_fn=lambda current, neighbor: 1, strategy='bfs')
```

This is the **core generic search engine** that implements three uninformed search strategies.

Key Components:

a) Parameters:

- `start_state`: Initial configuration (e.g., scrambled puzzle)
- `goal_test_fn`: Function to check if current state is the goal
- `successor_fn`: Function generating all possible next states
- `cost_fn`: Function computing transition cost (uniform 1 for BFS/DFS, variable for UCS)
- `strategy`: Search algorithm selection ('dfs', 'bfs', 'ucs')

b) Data Structures:

The implementation uses different data structures based on strategy:

```
if strategy == 'dfs':
    frontier = [(start_state, [], 0)]  # Stack (LIFO)
elif strategy == 'bfs':
    frontier = deque([(start_state, [], 0)])  # Queue (FIFO)
elif strategy == 'ucs':
    frontier = []  # Priority Queue (Min-Heap)
    heapq.heappush(frontier, (0, start_state, []))
```

- **DFS**: Uses Python list as stack (`.pop()` removes last element)
- **BFS**: Uses `collections.deque` for efficient O(1) popleft operations
- **UCS**: Uses `heapq` priority queue, ordered by cumulative path cost

c) Node Representation:

Each frontier node stores:

- **State**: Current puzzle configuration (tuple)
- **Path**: List of states from start to current (for solution reconstruction)
- **Cost (g)**: Cumulative path cost from start node

d) Search Loop Logic:

```
while frontier:
    # 1. Pop node based on strategy
    if strategy == 'dfs':
        current, path, g = frontier.pop()  # Last In, First Out
    elif strategy == 'bfs':
        current, path, g = frontier.popleft()  # First In, First Out
    else:  # ucs
        g, current, path = heapq.heappop(frontier)  # Minimum cost

    closed_nodes += 1

    # 2. Goal test
    if goal_test_fn(current):
        return path + [current]

    # 3. Skip if already visited (closed set optimization)
    if current in visited:
        continue

    visited.add(current)

    # 4. Expand successors
    for neighbor in successor_fn(current):
        if neighbor not in visited:
            new_cost = g + cost_fn(current, neighbor)

            # Add to frontier with updated path and cost
```

e) Performance Metrics:

The function tracks:

- **Open Nodes**: Total nodes added to frontier (measures branching)
- **Closed Nodes**: Total nodes expanded (measures search effort)

---

2. N-Puzzle Problem Formulation

The puzzle-specific logic is completely separated into utility functions:

a) Goal State Generation:

```
def create_goal_state(size):
    return tuple(list(range(1, size * size)) + [0])
```

For 8-puzzle (3×3): `(1, 2, 3, 4, 5, 6, 7, 8, 0)`
The blank tile is represented by `0`.

b) Successor Function:

```
def puzzle_successors(state, size):
    zero_pos = state.index(0)  # Find blank tile
    successors = []
    for move in get_moves(zero_pos, size):
        new_state = list(state)
        # Swap blank with adjacent tile
        new_state[zero_pos], new_state[move] = new_state[move], new_state[zero_pos]
        successors.append(tuple(new_state))
    return successors
```

Movement Logic:

```
def get_moves(position, size):
    moves = []
    row, col = divmod(position, size)  # Convert 1D index to 2D
    if row > 0: moves.append(position - size)  # up
    if row < size - 1: moves.append(position + size)  # down
    if col > 0: moves.append(position - 1)  # left
    if col < size - 1: moves.append(position + 1)  # right

    return moves
```

The blank tile can move up/down/left/right (2-4 possible moves depending on position).

c) Goal Test:

```
def puzzle_goal_test(state, size):

    return state == create_goal_state(size)
```

Simple equality check since goal state is deterministic.

---

3. Command-Line Interface

```
def parse_arguments():
    parser = argparse.ArgumentParser()
    parser.add_argument('--size', type=int, default=3)
    parser.add_argument('--start', type=str)
    parser.add_argument('--strategy', choices=['dfs', 'bfs', 'ucs'], default='bfs')

    return parser.parse_args()
```

Usage Examples:

```
bash
# Solve 8-puzzle with BFS (default)
python state_space_search.py

# Solve with custom initial state using DFS
python state_space_search.py --start "1,2,3,4,0,5,6,7,8" --strategy dfs

# Solve 15-puzzle (4×4) with UCS

python state_space_search.py --size 4 --start "1,2,3,4,5,6,7,8,9,10,11,12,13,14,0,15" --strategy ucs
```

---

4. Solution Visualization

```
def print_solution(solution, size):
    for step_num, step in enumerate(solution):
        print(f"Step {step_num}:")
        for i in range(size):
            row = step[i*size:(i+1)*size]

            print(' '.join(str(x) if x != 0 else '_' for x in row))
```

Converts 1D tuple representation to 2D grid visualization with _ representing the blank.

Search Strategy Comparison

Strategy	Frontier Structure	Completeness	Optimality	Time Complexity	Space Complexity
DFS	Stack (LIFO)	No (may get stuck in cycles)	No	O(b^m)	O(bm)
BFS	Queue (FIFO)	Yes (if finite)	Yes (for uniform cost)	O(b^d)	O(b^d)
UCS	Priority Queue	Yes	Yes	O(b^(1+⌊C*/ε⌋))	O(b^(1+⌊C*/ε⌋))

Where:

- **b** = branching factor (~2-4 for N-puzzle)
- **d** = depth of shallowest solution
- **m** = maximum depth of search tree
- **C*** = optimal solution cost
- **ε** = minimum step cost

Why This Design is Effective

1. Modularity:

The search algorithm knows nothing about puzzles—it only needs:

- Initial state
- Goal test function
- Successor generator

This means you can solve **any state-space problem** by just changing these three functions.

2. Reusability:

Want to solve a different problem? Just provide new functions:

```
# Example: Solving maze navigation
solution = uninformed_search(
    start_state=(0, 0),  # Starting position
    goal_test_fn=lambda s: s == (9, 9),  # Goal position
    successor_fn=get_valid_maze_moves,  # Your maze logic
    strategy='bfs'
)
```